

Calcul Numérique - Algèbre linéaire dense

T.DUFAUD

UVSQ

UNIVERSITÉ DE
VERSAILLES
ST-QUENTIN-EN-YVELINES



M1 CHPS - 5 / 11 / 2025

Rappel : Le calcul numérique et la simulation numérique

Algèbre linéaire : bases d'implémentation

Résolution de système linéaire

Système triangulaire
Méthode de Gauss
Factorisation LU
Pivot

1 Rappel : Le calcul numérique et la simulation numérique

2 Algèbre linéaire : bases d'implémentation

3 Résolution de système linéaire

- Système triangulaire
- Méthode de Gauss
- Factorisation LU
- Pivot

Rappel : Le
calcul
numérique et
la simulation
numérique

Algèbre
linéaire : bases
d'implémentation

Résolution de
système
linéaire

Système
triangulaire
Méthode de Gauss
Factorisation LU
Pivot

Rappel : Le calcul numérique et la simulation numérique

En quelques mots

- **Réaliser des expériences** sur des machines de calcul
- **Expériences dangereuses ou impossible à produire** (Nucléaire, Chimie, Astro-physique)
- **Utilisée dans l'industrie pour réduire les temps et les coûts de conception** (exemple : industrie automobile)
- **De plus en plus utilisée** (Nouveaux domaines d'applications, élargissement de la communauté scientifique (ex : Maison de la Simulation))
- **Activité pluridisciplinaire** (Domaine d'application, Modélisation, Schéma numérique, Algorithme de résolution (parallèle), Programmation (parallèle), Post-traitement)

Au coeur de la simulation

- **Intervient après la modélisation du problème**
- **Choix d'une méthode de résolution adaptée** : comprendre le problème, la manière dont le problème est posée contraint le choix de méthode (Propriétés des objets mathématiques à manipuler)
- **Étude des complexités et de la stabilité numérique** : Avoir un résultat juste pour un coût minimum (mémoire + temps CPU)
- **Évolution des architectures des machines de calcul** : les machines évoluent, les algorithmes doivent être adaptés ou bien inventés.

Plusieurs types de problèmes

- **Problème stationnaire** (ex : recherche de zéro par bisection)
- **Problème dynamique** (ex : chute d'un corps par Euler (EDO))
- **Problème discret linéaire** (ex : Écoulement stationnaire dans un milieu poreux, équation de Darcy : résolution par méthodes directes ou itératives)

Rappel : Le
calcul
numérique et
la simulation
numérique

Algèbre
linéaire : bases
d'implémentation

Résolution de
système
linéaire

Système
triangulaire
Méthode de Gauss
Factorisation LU
Pivot

Objectifs

- **Comprendre les contraintes du calcul en précision finie**
- **Implémenter un algorithme de calcul et l'analyser**
- **Améliorer un algorithme** : en termes de temps d'exécution, d'occupation mémoire et de stabilité numérique.
- **Utiliser l'algèbre linéaire pour la compréhension des problématiques du calcul numérique**

Rappel : Le
calcul
numérique et
la simulation
numérique

Algèbre
linéaire : bases
d'implémentation

Résolution de
système
linéaire

Système
triangulaire
Méthode de Gauss
Factorisation LU
Pivot

Algèbre linéaire : bases d'implémentation

Opérations vectorielles

Soient $a \in \mathbb{R}$, $x, y, z \in \mathbb{R}^n$.

- Multiplication d'un vecteur par un scalaire

$$z = ax \Rightarrow z_i = ax_i$$

- Addition de deux vecteurs

$$z = x + y \Rightarrow z_i = x_i + y_i$$

- Produit scalaire

$$a = x^T y \Rightarrow a = \sum_{i=1}^n x_i y_i$$

- Produit d'Hadamard (Terme à terme)

$$z = x \cdot y \Rightarrow z_i = x_i y_i$$

- SAXPY

$$y := ax + y \Rightarrow y_i = ax_i + y_i$$

Opérations matricielles

Soient $\alpha \in \mathbb{R}$. A , B et C des matrices réelles dont la taille est définie pour chaque opération.

- Transpose : $\mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{n \times m}$

$$C = A^T \Rightarrow c_{ij} = a_{ji}$$

- Addition : $\mathbb{R}^{m \times n} \times \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{m \times n}$

$$C = A + B \Rightarrow c_{ij} = a_{ij} + b_{ij}$$

- Multiplication par un scalaire : $\mathbb{R} \times \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{m \times n}$

$$C = \alpha A \Rightarrow c_{ij} = \alpha a_{ij}$$

- Multiplication Matrice-Matrice $\mathbb{R}^{m \times p} \times \mathbb{R}^{p \times n} \rightarrow \mathbb{R}^{m \times n}$

$$C = AB \Rightarrow c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}$$

Opérations Matrice-Vecteur

- GAXPY

$$y := Ax + y$$

Rappel : Le
calcul
numérique et
la simulation
numérique

Algèbre
linéaire : bases
d'implémentation

Résolution de
système
linéaire

Système
triangulaire

Méthode de Gauss
Factorisation LU
Pivot

Complexité et performances

- On identifie 3 ordres de complexité : linéaire, quadratique et cubique
- Chaque ordre correspond à un niveau : 1, 2 ou 3
- On essaye de remplacer les boucles par des algorithmes de niveau 1 2 ou 3.
- On peut faire appel à des bibliothèques optimisées (ex: BLAS)

Algorithm 1 Produit scalaire : $a = x^T y$

Require: $x, y \in \mathbb{R}^n$ $a = 0;$ **for** $i = 1 : n$ $a = a + x(i) * y(i);$ **end for**

Algorithm 2 Opération SAXPY : $y = ax + y$

Require: $a \in \mathbb{R}, x, y \in \mathbb{R}^n$ **for** $i = 1 : n$ $y(i) = a * x(i) + y(i);$ **end for**

Algorithm 3 Opération GAXPY double boucle, avec parcours prioritaire des lignes de A : $y = Ax + y$

Require: $x, y \in \mathbb{R}^n$ et $y = 0_n$
 for $i = 1 : m$
 for $j = 1 : n$
 $y(i) = A(i, j) * x(j) + y(i);$
 end for
 end for

Algorithm 4 Opération GAXPY double boucle, avec parcours prioritaire des colonnes de A : $y = Ax + y$

Require: $x, y \in \mathbb{R}^n$ et $y = 0_n$

```
for  $j = 1 : n$ 
  for  $i = 1 : m$ 
     $y(i) = A(i, j) * x(j) + y(i);$ 
  end for
end for
```

Algorithm 5 Opération GAXPY une boucle, avec parcours prioritaire des lignes de A : $y = Ax + y$

Require: $x, y \in \mathbb{R}^n$ et $y = 0_n$
 for $i = 1 : m$
 $y(i) = A(i, :) * x + y(i);$
 end for

Algorithm 6 Opération GAXPY une boucle, avec parcours prioritaire des colonnes de A : $y = Ax + y$

Require: $x, y \in \mathbb{R}^n$ et $y = 0_n$
 for $j = 1 : n$
 $y = A(:, j) * x(j) + y;$
 end for

Algorithm 7 Produit Matrice-Matrice "ijk"

Require: $A \in \mathbb{R}^{m \times p}$, $B \in \mathbb{R}^{p \times n}$ et $C \in \mathbb{R}^{m \times n}$

```
for  $i = 1 : m$ 
  for  $j = 1 : n$ 
    for  $k = 1 : p$ 
       $C(i, j) = A(i, k)B(k, j) + C(i, j);$ 
    end for
  end for
end for
```

Rappel : Le calcul numérique et la simulation numérique

Algèbre linéaire : bases d'implémentation

Résolution de système linéaire

Système triangulaire

Méthode de Gauss

Factorisation LU

Pivot

Version	Boucle intermédiaire	Boucle intérieure	Accès aux données (boucle intérieure)
ijk	vecteur $\times$ matrice	produit scalaire	ligne de A et colonne de B
jik	matrice $\times$ vecteur	produit scalaire	ligne de A et colonne de B
ikj	GAXPY ligne	SAXPY	ligne de B et ligne de C
jki	GAXPY colonne	SAXPY	colonne de A et colonne de C
kij	produit extérieur de lignes	SAXPY	ligne de B et ligne de C
kji	produit extérieur de colonnes	SAXPY	colonne de A et colonne de C

Table: Opérations des boucles selon la version d'algorithme du produit Matrice-Vecteur

Algorithm 8 Produit Matrice-Matrice "ijk" avec 2 boucles

Require: $A \in \mathbb{R}^{m \times p}$, $B \in \mathbb{R}^{p \times n}$ et $C \in \mathbb{R}^{m \times n}$ **for** $i = 1 : m$ **for** $j = 1 : n$ $C(i, j) = A(i, :)B(:, j) + C(i, j);$ **end for****end for**

Algorithm 9 Produit Matrice-Matrice "ijk" avec 1 boucle

Require: $A \in \mathbb{R}^{m \times p}$, $B \in \mathbb{R}^{p \times n}$ et $C \in \mathbb{R}^{m \times n}$ **for** $i = 1 : m$ $C(i, :) = A(i, :)B + C(i, :);$ **end for**

Rappel : Le
calcul
numérique et
la simulation
numérique

Algèbre
linéaire : bases
d'implémentation

Résolution de
système
linéaire

Système
triangulaire

Méthode de Gauss

Factorisation LU

Pivot

Résolution de système linéaire

Système linéaire

Soient $A \in \mathbb{R}^{n \times n}$, $x, b \in \mathbb{R}^{n \times 1}$, A inversible, i.e. $\det(A) \neq 0$. Dans la suite, on souhaite résoudre le système linéaire suivant :

$$Ax = b \quad (1)$$

Erreur avant, arrière et conditionnement

- **Erreur avant :**

$$err := \frac{\|x - \hat{x}\|}{\|x\|} \quad (2)$$

- **Erreur arrière :** le résidu $r \in \mathbb{R}^n$, $r = b - A\hat{x}$. Soit l'erreur résiduelle relative $relres \in \mathbb{R}$:

$$relres := \frac{\|b - A\hat{x}\|}{\|A\| \|\hat{x}\|} \quad (3)$$

- **Conditionnement de A :** $\kappa(A) = \|A\| \|A^{-1}\|$. Le conditionnement peut être vu comme la capacité pour A à générer de l'erreur
- **Règle générale :**

$$\frac{\|x - \hat{x}\|}{\|x\|} \leq \kappa_2(A) \frac{\|b - A\hat{x}\|}{\|A\| \|\hat{x}\|} \quad (4)$$

Problématique

- Quelle méthode choisir ?
- De quelle précision avons-nous besoin ?
- Quelles sont les contraintes matérielles ? (espace mémoire, architecture)
- Quel type de matrice doit-on inverser ?
- Y a-t-il des propriétés intéressantes à exploiter ?

L'idée des méthodes directes

- Transformer un système en un système équivalent plus simple à résoudre.
- Les systèmes triangulaires sont simples à résoudre !

Matrice triangulaire

Definition 1.

[Matrice triangulaire supérieure] La matrice $U \in \mathbb{R}^{n \times n}$, est triangulaire supérieure si $\forall i > j, u_{ij} = 0$. U est donc de la forme :

$$U = \begin{pmatrix} X & X & X & X & X \\ & X & X & X & X \\ & & X & X & X \\ & & & X & X \\ & & & & X \end{pmatrix}$$

Definition 2 (Matrice triangulaire inférieure).

La matrice $L \in \mathbb{R}^{n \times n}$, est triangulaire inférieure si $\forall i < j, l_{ij} = 0$. L est donc de la forme :

$$L = \begin{pmatrix} X & & & & \\ X & X & & & \\ X & X & X & & \\ X & X & X & X & \\ X & X & X & X & X \end{pmatrix}$$

Algorithm 10 Méthode de remontée : résolution de $Ux = b$ (version avec produit scalaire)

Require: $U \in \mathbb{R}^{n \times n}$, $x, b \in \mathbb{R}^n$

$x(n) = b(n) / U(n, n);$

for $i = n - 1 : -1 : 1$

$x(i) = (b(i) - U(i, (i + 1) : n) * x((i + 1) : n)) / U(i, i);$

end for

Algorithm 11 Méthode de descente : résolution de $Lx = b$ (version avec produit scalaire)

Require: $L \in \mathbb{R}^{n \times n}$, $x, b \in \mathbb{R}^n$

$x(1) = b(1) / L(1, 1);$

for $i = 2 : n$

$x(i) = (b(i) - L(i, 1 : (i - 1)) * x(1 : (i - 1))) / L(i, i);$

end for

Principe général

La méthode de Gauss s'applique en deux étapes :

- 1 Triangularisation du système tel que

$$Ax = b \Leftrightarrow MAX = Mb$$

ou MA est une matrice triangulaire supérieure.

- 2 Résolution de $MAX = Mb$ par "remontée".

La triangularisation, **élimination de Gauss**, s'effectue en $n - 1$ étapes et consiste à mettre à zéro les éléments d'une colonne situés sous la diagonale principale.

Initialisation et Élimination 1

- Initialisation : $A^{(1)} := A$ et $b^{(1)} := b$ et pivot $d_1 = a_{11}^{(1)}$
- Pour $k = 1$, $d_1 = a_{11}^{(1)}$

$$\begin{aligned}
 A^{(2)} &= M_1 A^{(1)} \\
 &= \begin{pmatrix} 1 & & & \\ -m_{21} & 1 & & \\ -m_{31} & & 1 & \\ \vdots & & & \ddots \\ -m_{n1} & & & & 1 \end{pmatrix} A^{(1)}, \text{ où } m_{(k+1):n,1} = A^{(1)}(2:n,1)/d_1 \\
 &= \begin{pmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \dots & a_{1n}^{(1)} \\ 0 & a_{22}^{(2)} & \dots & a_{2n}^{(2)} \\ \vdots & \vdots & & \vdots \\ 0 & a_{n2}^{(2)} & \dots & a_{nn}^{(2)} \end{pmatrix}
 \end{aligned}$$

et,

$$b^{(2)} = M_1 b^{(1)}$$

$k = 2 : n - 2$ éliminations

- Pour $1 < k < n - 1$, $d_k = a_{kk}^{(k)}$

$$\begin{aligned}
 A^{(k+1)} &= M_k A^{(k)} \\
 &= \begin{pmatrix} 1 & & & & & & & & & \\ & \ddots & & & & & & & & \\ & & 1 & & & & & & & \\ & & -m_{(k+1)k} & 1 & & & & & & \\ & & \vdots & & \ddots & & & & & \\ & & -m_{nk} & & & 1 & & & & \end{pmatrix} A^{(k)}, \text{ où } m_{(k+1):n,1} = A^{(k)}((k+1) : n, k) / d_k \\
 &= \begin{pmatrix} a_{11}^{(k)} & a_{12}^{(k)} & \dots & \dots & \dots & \dots & \dots & \dots & a_{1n}^{(k)} \\ 0 & a_{22}^{(k)} & a_{23}^{(k)} & \dots & \dots & \dots & \dots & \dots & a_{2n}^{(k)} \\ & 0 & 0 & \ddots & & & & & \\ \vdots & \vdots & & & a_{kk}^{(k)} & a_{(k+1)k}^{(k)} & \dots & \dots & a_{kn}^{(k)} \\ \vdots & \vdots & & & 0 & a_{(k+1)k}^{(k+1)} & \dots & \dots & a_{(k+1)n}^{(k+1)} \\ \vdots & \vdots & & & \vdots & \vdots & & & \vdots \\ 0 & 0 & & & 0 & a_{nk}^{(k+1)} & \dots & \dots & a_{nn}^{(k+1)} \end{pmatrix}
 \end{aligned}$$

et,

$$b^{(k+1)} = M_k b^{(k)}$$

$k = 2 : n - 1$ éliminations

- Pour $k = n - 1$, on obtient $A^{(n)}$ triangulaire supérieure telle que :

$$A^{(n)} = M_{n-1}A^{(n-1)} = MA$$

et,

$$b^{(n)} = Mb$$

Avec,

$$\begin{aligned} M &= M_{n-1}M_{n-2} \dots M_1 \\ &= \begin{pmatrix} 1 & & & & \\ -m_{21} & 1 & & & \\ \vdots & & \ddots & & \\ \vdots & & & \ddots & \\ -m_{n1} & -m_{n2} & \dots & -m_{n(n-1)} & 1 \end{pmatrix} \end{aligned} \quad (5)$$

Algorithm 12 Méthode de résolution de Gauss sans pivotage : résolution de $Ax = b$ (version scalaire à 3 boucles "kij")

Require: $A \in \mathbb{R}^{n \times n}$, $x, b \in \mathbb{R}^n$

```
for  $k = 1 : n - 1$ 
  for  $i = k + 1 : n$ 
     $m_{ik} = A(i, k) / A(k, k);$ 
     $b(i) = b(i) - m_{ik} * b(k);$ 
    for  $j = k + 1 : n$ 
       $A(i, j) = A(i, j) - m_{ik} * A(k, j);$ 
    end for
  end for
end for
```

Résoudre le nouveau système $Ax = b$ par remontée (Algorithme 10).

- La méthode est dépendante du second membre.
- Complexité de l'élimination : $2n^3/3$ flops

De Gauss à LU

Posons $U = MA$. On peut écrire $A = LU$ avec L triangulaire inférieur.
En considérant l'écriture de M donnée par l'équation (6) on a M inversible, d'où :

$$U = MA \Leftrightarrow M^{-1}U = A$$

$$\text{et } M^{-1} = M_1^{-1}M_2^{-1} \dots M_{n-2}^{-1}M_{n-1}^{-1}$$

On obtient alors

$$M^{-1} = \begin{pmatrix} 1 & & & & \\ m_{21} & 1 & & & \\ \vdots & m_{32} & 1 & & \\ \vdots & \vdots & & \ddots & \\ m_{n1} & m_{n2} & \dots & m_{n(n-1)} & 1 \end{pmatrix} \quad (6)$$

On a donc bien une matrice triangulaire M^{-1} et on pose $L = M^{-1}$.
La méthode de Gauss donne donc une factorisation LU de la matrice.

Principe général

1 Factorisation $A = LU$

2 Résolution de

$$\begin{cases} Ly = b \\ Ux = y \end{cases} \quad (7)$$

Forme compacte de LU

$$A = LU := \begin{pmatrix} u_{11} & \dots & \dots & \dots & u_{1n} \\ l_{21} & u_{22} & \dots & \dots & u_{2n} \\ l_{31} & l_{32} & \ddots & \dots & \dots \\ \vdots & \vdots & \ddots & \ddots & u_{(n-1)n} \\ l_{n1} & l_{n2} & \dots & l_{n(n-1)} & u_{nn} \end{pmatrix} \quad (8)$$

Algorithm 13 Élimination de Gauss et écriture compacte de LU (version scalaire à 3 boucles "kij")

Require: $A \in \mathbb{R}^{n \times n}$, $x, b \in \mathbb{R}^n$

```
for  $k = 1 : n - 1$ 
  for  $i = k + 1 : n$ 
     $A(i, k) = A(i, k) / A(k, k);$ 
  end for
  for  $i = k + 1 : n$ 
    for  $j = k + 1 : n$ 
       $A(i, j) = A(i, j) - A(i, k) * A(k, j);$ 
    end for
  end for
end for
```

Comme pour le produit Matrice-Matrice il y a 6 écritures possibles.

Problématique

Soit à résoudre le système linéaire (1) par l'algorithme 12. Deux situations critiques peuvent apparaître :

- 1 si $d_k = a_{kk}^{(k)} = 0$ alors l'opération d'élimination n'est pas définie (division par zéro).
- 2 si $d_k = a_{kk}^{(k)} = \epsilon \ll 1$

Exemple pour le cas 2

- $A = \begin{pmatrix} \epsilon & -1 \\ 1 & 1 \end{pmatrix}$ avec $0 < \epsilon \ll 1$
- $\kappa_{\infty}(A) = 4/(1 + \epsilon)$
- Pour la factorisation LU on peut écrire :

$$\begin{aligned} A &= \begin{pmatrix} \epsilon & -1 \\ 1 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ l_{21} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ \epsilon^{-1} & 1 \end{pmatrix} \begin{pmatrix} \epsilon & -1 \\ 0 & \epsilon^{-1} \end{pmatrix} \end{aligned}$$

D'où,

$$A - \hat{L}\hat{U} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$$

Le problème est dans le choix du pivot et non dans le conditionnement de A .

Pivot partiel

Prendre pour pivot le plus grand élément de la colonne en valeur absolue.
C'est-à-dire, à l'étape k on échange la ligne k avec la ligne r tel que :

$$|a_{rk}^{(k)}| = \max_{k \leq i \leq n} (a_{ik}^{(k)}) \quad (9)$$

Ceci assure que

$$|m_{ik}| \leq 1, \text{ pour } i = k + 1 : n \quad (10)$$

Cette méthode dite du pivot partiel à une complexité en $O(n^2)$.

Pivot complet

On pourrait, pour assurer une plus grande stabilité, échanger à l'étape k les lignes r et les colonnes s de A tel que :

$$|a_{rs}^{(k)}| = \max_{k \leq i \leq n, k \leq j \leq n} (a_{ij}^{(k)}) \quad (11)$$

Cette méthode dite du pivot complet à une complexité en $O(n^3)$.

Algorithm 14 Pivot Partiel

Require: $A \in \mathbb{R}^{n \times n}$ à l'étape k de l'élimination de Gauss,
 $row \in \mathbb{R}^{1 \times n}$ le vecteur des permutations des lignes de A
 q la ligne de A à permuter à cette étape

- 1: $[piv, ind] = \max(abs(A(k : n, k)))$; // **récupère le max piv à la ligne ind du vecteur colonne sous l'élément diagonal $A(k, k)$**
 - 2: $ind = k - 1 + ind$; // **convertit l'indice local en un indice global de la ligne à échanger.**
 - 3: $q = row(1, ind)$;
 - 4: **if** $ind \neq k$ **then**
 - 5: $new = A(ind, :)$; // **stockage temporaire de la ligne à échanger dans le vecteur new**
 - 6: $A(ind, :) = A(k, :)$; // **échange**
 - 7: $row(1, ind) = row(1, ind)$; et $row(1, k) = q$; // **mise à jour du vecteur des permutations**
 - 8: **end if**
 - 9: **end**
-

Rappel : Le
calcul
numérique et
la simulation
numérique

Algèbre
linéaire : bases
d'implémentation

Résolution de
système
linéaire

Système
triangulaire
Méthode de Gauss
Factorisation LU

Pivot

$$PA = LU$$

On écrit généralement la matrice des permutations, $P \in \mathbb{R}^{n \times n}$ après les k étapes d'élimination.

Lorsque l'on calcule une factorisation LU avec pivot partiel on obtient :

$$PA = LU$$

Cela revient à résoudre le système :

$$Ax = b \Leftrightarrow LUx = Pb$$